[Dung Vu] Userspace Debugging Toolkit: GDB, strace & Valgrind

Section: Linux System Programming

Topic: Userspace Debugging Toolkit: GDB, strace & Valgrind

---

1. GDB

GDB, the GNU Project debugger, allows you to see what is going on `inside' another program while it executes - or what another program was doing at the moment it crashed.

When compiling a C program, it should be compiled with the flag `-g`:

- Without `-g`:
  - The executable mainly contains the machine code.
  - GDB can still debug the machine instructions, but the debugging becomes much more low-level.
- With `-g`:
  - GCC adds debugging information to the executable.

```
machine code
+
debug information
        |
        ├── address → source file + line
        ├── address → function
        ├── variable → type/location
        └── source-level information
```

  - The debugging information gives translation between machine-level execution and source-level concepts.

1.1. Debugger concepts

Breakpoints: pause execution at a specific point

Conditional breakpoints: pause execution at a specific point when a condition is hit

Continue: run until next breakpoint

Watchpoints: pause execution when a memory location is accessed (write/read)

Stepping:

- ***Step (into)***: execute one source line; if it's a call, follow into the callee.
- ***Next (over)***: execute one source line; if it's a call, run it to completion without stopping inside
- ***Finish (out)***: run until the current function returns

1.2. Cheatsheet

gdb_cheat_sheet_dark.svg

1.3. Stack traces

A stack trace (backtrace) is a report of the active stack frames at a certain point in time during the execution of a program.

Example:

```
#include <stdio.h>

void demo3()
{
    printf("Hello");
}

void demo2()
{
    demo3();
}

void demo1()
```

```
{
    demo2();
}

void demo()
{
    demo1();
}

int main()
{
    demo();
    return 0;
}
```

GDB:

Set a breakpoint at line 5:

```
(gdb) b 5
```

Run and do a backtrace:

```
(gdb) run
Breakpoint 1, demo3 () at backtrace.c:5
5           printf("Hello");
(gdb) backtrace
#0  demo3 () at backtrace.c:5
#1  0x0000555555555175 in demo2 () at backtrace.c:10
#2  0x0000555555555185 in demo1 () at backtrace.c:15
#3  0x0000555555555195 in demo () at backtrace.c:20
#4  0x00005555555551a5 in main () at backtrace.c:25
```

1.4. Core dumps

A core dump consists of the recorded state of the working memory of a computer program at a specific time, generally when the program has crashed or otherwise terminated abnormally.

Setup by remove the shell's size limit on core dumps.

```
$ ulimit -c unlimited # Set core-dump limit to unlimited
$ ulimit -c # Shows the current core-dump limit
unlimited
```

Load a core dump with GDB by:

```
$ gdb <executable> <coredump>
```

1.3. Multi-thread debugging

Thread Inspection

Command	Effect
`info threads`	List all threads, current one marked with *
`thread <N>`	Switch focus to thread `N` (GDB's own numbering)
`thread apply <N> <cmd>`	Run `<cmd>` in thread N without switching
`thread apply all <cmd>`	Run `<cmd>` in every thread
`thread apply all bt`	Backtrace of every thread — most-used command in deadlock/hang debugging
`thread find <regex>`	Find threads whose name/function matches regex
`thread name <name>`	Assign a name to current thread (cosmetic)

Breakpoints per Thread

```
break worker.c:20 thread 2          # only stops when thread 2 hits it
break foo if thread == 3            # equivalent, condition form
condition 1 thread == 2             # add thread condition to existing bp N=1
```

Combine with a normal condition:

```
break foo thread 2 if x > 10
```

Watchpoints work the same way and are thread-aware:

```
watch shared_counter thread 2
```

1.4. Multi-process debugging

2. strace

`strace` is a Linux command-line tool that intercepts and records system calls and signals made by a process to debug and analyze its behavior.

Flag	Purpose
`-f`	Follow forked child processes/threads
`-ff`	Like `-f`, but writes each process's trace to a separate file (`-o` becomes a prefix)
`-p <PID>`	Attach to an already-running process instead of launching one
`-o <file>`	Write output to a file instead of `stderr`
`-D`	Run the tracer as a grandchild, not a direct parent
`-tt`	Print timestamp before each line
`-T`	Show time spent inside each syscall
`-r`	Print relative timestamp (time since previous line) instead of absolute
`-e trace=open,read,write`	Only show these syscalls
`-e trace=file`	All file-related syscalls (open, stat, access, chdir...)
`-e trace=network`	All socket/network syscalls
`-e trace=process`	fork, clone, execve, exit, wait...
`-e trace=signal`	Signal delivery/handling
`-e trace=memory`	mmap, brk, etc.
`-e trace=desc`	File-descriptor-related calls (read/write/close/dup/select/poll...)
`-e trace=%stat`	Just `stat`-family calls
`-Z` `-e status=failed`	Only print syscalls that failed
`-e signal=!SIGCHLD`	Exclude noisy signals (e.g. `SIGCHLD` spam)

3. Valgrind

Valgrind is an instrumentation framework for building dynamic analysis tools. It comes with a set of tools each of which performs some kind of debugging, profiling, or similar task that helps improve

programs.

A number of useful tools are supplied in Valgrind:

1. *Memcheck*: a memory error detector
2. *Cachegrind*: a cache and branch-prediction profiler
3. *Callgrind*: a call-graph generating cache profiler
4. *Helgrind*: a thread error detector
5. *DRD*: a thread error detector
6. *Massif*: a heap profiler
7. *DHAT*: a heap profiler
8. *BBV*: an experimental SimPoint basic block vector generator

In this section, we only focus on `Memcheck`

4.1. Memory leaks

Memory leaks occur when we allocate memory but never free it.

Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char *str = malloc(20);
    strcpy(str, "Hello, World!");
    printf("%s\n", str);

    return 0;
}
```

```
$ valgrind --leak-check=full ./mem_leak
...
==58861== HEAP SUMMARY:
==58861==     in use at exit: 20 bytes in 1 blocks
==58861==   total heap usage: 2 allocs, 1 frees, 1,044 bytes allocated
==58861==
==58861== 20 bytes in 1 blocks are definitely lost in loss record 1 of 1
==58861==    at 0x4850858: malloc (vg_replace_malloc.c:447)
```

```
==58861==    by 0x400117E: main (mem_leak.c:7)
==58861==
==58861== LEAK SUMMARY:
==58861==    definitely lost: 20 bytes in 1 blocks
==58861==    indirectly lost: 0 bytes in 0 blocks
==58861==      possibly lost: 0 bytes in 0 blocks
==58861==    still reachable: 0 bytes in 0 blocks
==58861==         suppressed: 0 bytes in 0 blocks
...
```

Fix:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char *str = malloc(20);
    strcpy(str, "Hello, World!");
    printf("%s\n", str);

    free(str); /* Free after use */
    return 0;
}
```

```
$ valgrind --leak-check=full ./mem_leak
...
==58803== HEAP SUMMARY:
==58803==     in use at exit: 0 bytes in 0 blocks
==58803==   total heap usage: 2 allocs, 2 frees, 1,044 bytes allocated
==58803==
==58803== All heap blocks were freed -- no leaks are possible
...
```

4.2. Invalid reads/writes

Invalid reads/writes occur when:

- Accessing memory outside the bound of what was allocated.

- Deferencing a pointer that doesn't point to anything meaningful

Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char *str = malloc(10);
    strcpy(str, "0123456789");

    free(str);
    return 0;
}
```

```
$ valgrind -s ./invalid_rw
...
==8706== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
==8706==
==8706== 1 errors in context 1 of 1:
==8706== Invalid write of size 4
==8706==    at 0x4001194: main (invalid_rw.c:8)
==8706==  Address 0x4a91047 is 7 bytes inside a block of size 10 alloc'd
==8706==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==8706==    by 0x400117E: main (invalid_rw.c:7)
==8706==
==8706== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

4.3. Use-after-free

Use-after-free occurs when we keep using the pointer after having freed it

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
```

```
    int *i = malloc(sizeof(int));
    *i = 10;

    free(i);

    printf("%d\n", *i); /* Use after free */
    return 0;
}
```

```
$ valgrind ./use_after_free
...
==10272== Invalid read of size 4
==10272==    at 0x40011BD: main (use_after_free.c:12)
==10272==  Address 0x4a91040 is 0 bytes inside a block of size 4 free'd
==10272==    at 0x48538BF: free (vg_replace_malloc.c:990)
==10272==    by 0x40011B8: main (use_after_free.c:10)
==10272==  Block was alloc'd at
==10272==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==10272==    by 0x400119E: main (use_after_free.c:7)
...
```

`Block was alloc'd` tells that the block had been alloc'd before then got freed

4.4. Uninitialized memory

Example:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *i = malloc(sizeof(int));

    printf("%d\n", *i); /* Use of uninitialized memory */

    free(i);
    return 0;
}
```

```
$ valgrind --track-origins=yes ./uninitialized
...
==12857== Use of uninitialised value of size 8
==12857==    at 0x48D0062: _itoa_word (_fitoa_word.c:38)
==12857==    by 0x48DB61C: __printf_buffer (vfprintf-process-arg.c:155)
==12857==    by 0x48DD767: __vfprintf_internal (vfprintf-internal.c:1548)
==12857==    by 0x48D1132: printf (printf.c:33)
==12857==    by 0x40011BE: main (uninitialized.c:8)
==12857==  Uninitialised value was created by a heap allocation
==12857==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==12857==    by 0x400119E: main (uninitialized.c:6)
...
```

4. Lab

Lab 1

Compile a program with a deliberately introduced segfault; debug it live with GDB (breakpoints/backtrace) and then again by loading a generated core dump

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

char *buf;

int sum_to_n(int num)
{
    int i, sum = 0;
    for (i = 1; i <= num; i++)
        sum += i;
    return sum;
}

void printSum()
{
    char line[10];
```

```
    printf("enter a number:\n");
    fgets(line, 10, stdin);
    if (line != NULL)
        strtok(line, "\n");
    int sum = sum_to_n(atoi(line));
    sprintf(buf, "sum=%d", sum);
    printf("%s\n", buf);
}

int main(void)
{
    printSum();
    return 0;
}
```

```
$ gcc -g lab_1.c -o lab_1
```

Output:

```
$ ./lab_1
enter a number:
5
Segmentation fault         ./lab_1
```

After setting up the core-dump limit, the kernel can produce a core dump for our program when segfault (`core.11314` in this case)

```
$ ls | grep core
core.11314
```

Load the core dump with GDB:

```
$ gdb ./lab_1 ./core.11314
...
Core was generated by `./lab_1'.
Program terminated with signal SIGSEGV, Segmentation fault.
#0  __memcpy_evex_unaligned_erms () at ../sysdeps/x86_64/multiarch/memmove-vec-unaligned-
erms.S:289
...
```

Do a backtrace:

```
(gdb) bt
#0  __memcpy_evex_unaligned_erms () at ../sysdeps/x86_64/multiarch/memmove-vec-unaligned-
erms.S:289
#1  0x000075d49b264d68 in __GI_memcpy (__dest=<optimized out>, __src=<optimized out>, __len=4)
    at ../string/bits/string_fortified.h:29
#2  __printf_buffer_write (buf=buf@entry=0x7fff4086e040, s=<optimized out>,
s@entry=0x60c487aaa016 "sum=%d",
    count=count@entry=4) at ./stdio-common/Xprintf_buffer_write.c:39
#3  0x000075d49b26dacd in __printf_buffer (buf=buf@entry=0x7fff4086e040, format=0x60c487aaa016
"sum=%d",
    ap=ap@entry=0x7fff4086e090, mode_flags=mode_flags@entry=0) at ./stdio-common/vfprintf-
internal.c:652
#4  0x000075d49b28f76c in __vsprintf_internal (string=<optimized out>,
maxlen=maxlen@entry=18446744073709551615,
    format=<optimized out>, args=args@entry=0x7fff4086e090, mode_flags=mode_flags@entry=0)
    at ./libio/iovsprintf.c:62
#5  0x000075d49b26a887 in __sprintf (s=<optimized out>, format=<optimized out>) at ./stdio-
common/sprintf.c:30
#6  0x000060c487aa92aa in printSum () at lab_1.c:23
#7  0x000060c487aa92dd in main () at lab_1.c:29
```

In this backtrace, frame #6 shows the crash originates from our own code at `lab_1.c:23`:

```
sprintf(buf, "sum=%d", sum);
```

Furthermore, on the upper stack frame (#0-#5), there was a problem with some kind of `memcpy` action => maybe `sprintf` failed to copy data to `buf`. Let's investigate the `char *buf` variable.

```
(gdb) print buf
$1 = 0x0
```

=> That's the problem. The pointer `buf` was pointed to `NULL` => `buf` wasn't got allocated.

Lab 2

Debug a multi-threaded program with GDB (thread apply all bt) to locate a race-condition-induced crash

```
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <pthread.h>

#define NUM_THREADS 2
#define OPS_PER_THREAD 200000

typedef struct {
    int *data;
    int size;
    int capacity;
} DynArray;

DynArray arr;

void arr_push(DynArray *a, int value) {
    if (a->size == a->capacity) {
        a->capacity *= 2;
        a->data = realloc(a->data, a->capacity * sizeof(int));
    }
    a->data[a->size] = value;
    a->size++;
}

void *worker(void *arg) {
    long id = (long)arg;
    for (int i = 0; i < OPS_PER_THREAD; i++) {
        arr_push(&arr, (int)id);
    }
    return NULL;
}

int main(void) {
    arr.capacity = 4;
    arr.size = 0;
    arr.data = malloc(arr.capacity * sizeof(int));

    pthread_t threads[NUM_THREADS];
    for (long i = 0; i < NUM_THREADS; i++)
        pthread_create(&threads[i], NULL, worker, (void *)i);

    for (int i = 0; i < NUM_THREADS; i++)
```

```
        pthread_join(threads[i], NULL);

    printf("Final size: %d (expected %d)\n", arr.size, NUM_THREADS * OPS_PER_THREAD);

    long sum = 0;
    for (int i = 0; i < arr.size; i++)
        sum += arr.data[i];
    printf("Sum: %ld\n", sum);

    free(arr.data);
    return 0;
}
```

Output:

```
$ ./lab_2
free(): invalid next size (normal)
Segmentation fault         (core dumped) ./lab_2
```

Debug with core dump:

```
(gdb) thread apply all bt

Thread 3 (Thread 0x7c97694b3740 (LWP 60216)):
#0  __syscall_cancel_arch () at ../sysdeps/unix/sysv/linux/x86_64/syscall_cancel.S:56
#1  0x00007c97692a030c in __internal_syscall_cancel (a1=a1@entry=136989745609960,
a2=<optimized out>, a3=<optimized out>, a4=a4@entry=0, a5=a5@entry=0, a6=a6@entry=4294967295,
nr=202) at ./nptl/cancellation.c:49
#2  0x00007c97692a06e7 in __futex_abstimed_wait_common64 (private=128,
futex_word=0x7c97691ffce8, expected=<optimized out>, op=265, abstime=0x0, cancel=true) at
./nptl/futex-internal.c:57
#3  __futex_abstimed_wait_common (futex_word=0x7c97691ffce8, expected=<optimized out>,
clockid=0, abstime=0x0, private=128, cancel=true) at ./nptl/futex-internal.c:87
#4  __GI___futex_abstimed_wait_cancelable64 (futex_word=futex_word@entry=0x7c97691ffce8,
expected=<optimized out>, clockid=clockid@entry=0, abstime=abstime@entry=0x0,
private=private@entry=128) at ./nptl/futex-internal.c:139
#5  0x00007c97692a5dd9 in __pthread_clockjoin_ex (threadid=136989745608384, thread_return=0x0,
clockid=0, abstime=0x0, cancel=<optimized out>) at ./nptl/pthread_join_common.c:66
#6  0x0000580dab53f38d in main () at lab_2.c:43
```

```
Thread 2 (Thread 0x7c97689fe6c0 (LWP 60218)):
#0  0x0000580dab53f283 in arr_push (a=0x580dab542020 <arr>, value=1) at lab_2.c:21
#1  0x0000580dab53f2d1 in worker (arg=0x1) at lab_2.c:28
#2  0x00007c97692a3dfa in start_thread (arg=<optimized out>) at ./nptl/pthread_create.c:454
#3  0x00007c97693375cc in __GI___clone3 () at ../sysdeps/unix/sysv/linux/x86_64/clone3.S:78

Thread 1 (Thread 0x7c97691ff6c0 (LWP 60217)):
#0  __pthread_kill_implementation (threadid=<optimized out>, signo=6, no_tid=0) at
./nptl/pthread_kill.c:44
#1  __pthread_kill_internal (threadid=<optimized out>, signo=6) at ./nptl/pthread_kill.c:89
#2  __GI___pthread_kill (threadid=<optimized out>, signo=signo@entry=6) at
./nptl/pthread_kill.c:100
#3  0x00007c9769245b7e in __GI_raise (sig=sig@entry=6) at ../sysdeps/posix/raise.c:26
#4  0x00007c97692288ec in __GI_abort () at ./stdlib/abort.c:77
#5  0x00007c9769229979 in __libc_message_impl (vma_name=vma_name@entry=0x7c97693db569 "glibc:
fatal", fmt=fmt@entry=0x7c97693debbe "%s\n") at ../sysdeps/posix/libc_fatal.c:138
#6  0x00007c97692b08ac in __libc_message_wrapper (vmaname=0x7c97693db569 "glibc: fatal",
fmt=0x7c97693debbe "%s\n") at ../include/stdio.h:203
#7  malloc_printerr (str=str@entry=0x7c97693e1648 "free(): invalid next size (normal)") at
./malloc/malloc.c:5341
#8  0x00007c97692b2254 in _int_free_merge_chunk (av=0x7c9769412ac0 <main_arena>,
p=0x580db618e260, size=131088) at ./malloc/malloc.c:4428
#9  0x00007c97692b2329 in _int_free_chunk (av=<optimized out>, p=<optimized out>,
size=<optimized out>, have_lock=<optimized out>) at ./malloc/malloc.c:4368
#10 0x00007c97692b4379 in _int_realloc (av=av@entry=0x7c9769412ac0 <main_arena>,
oldp=oldp@entry=0x580db618e260, oldsize=131088, nb=262160) at ./malloc/malloc.c:4642
#11 0x00007c97692b579f in __GI___libc_realloc (oldmem=0x580db618e270, bytes=<optimized out>)
at ./malloc/malloc.c:3480
#12 0x0000580dab53f262 in arr_push (a=0x580dab542020 <arr>, value=0) at lab_2.c:19
#13 0x0000580dab53f2d1 in worker (arg=0x0) at lab_2.c:28
#14 0x00007c97692a3dfa in start_thread (arg=<optimized out>) at ./nptl/pthread_create.c:454
#15 0x00007c97693375cc in __GI___clone3 () at ../sysdeps/unix/sysv/linux/x86_64/clone3.S:78
```

The log shows that:

- Thread 3 (main thread), thread 1 + 2 (2 worker threads)
- Thread 1 stopped at `lab_2.c:19` (frame #12):

  ```
  19 a->data = realloc(a->data, a->capacity * sizeof(int));
  ```

- Thread 2 stopped at `lab_2.c:21` (frame #0):

```
21 a->data[a->size] = value;
```

- Thread 1 failed: It called `abort` (frame #4) after trying to `free` the old array (frame #9)

=> Write operation in thread 2 collided with instructions of `realloc()` in thread 1, causing it to fail

=> Solution: A mutex lock should be used to protect this critical section `arr_push(&arr, (int)id);`

Lab 3

Run strace on a program that hangs or fails silently; use the syscall trace (files/sockets/processes/permissions/signals) to find the root cause

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    int filedes[2];

    pipe(filedes);

    switch (fork())
    {
        case -1:
            perror("fork");
            return -1;

        case 0: /* Child process */
            // close(filedes[1]); /* Doesn't close unused write end */

            /* Child now reads from pipe */
            char buffer[100];
            while(read(filedes[0], buffer, sizeof(buffer)) > 0)
            {
                printf("Received from parent: %s\n", buffer);
            }

            close(filedes[0]); /* Close read end after writing */
            break;
```

```
        default: /* Parent process */
            close(filedes[0]); /* Close unused read end */

            /* Parent now writes to pipe */
            write(filedes[1], "Hello, child!", 14);

            close(filedes[1]); /* Close write end after reading */
            break;
    }

    return 0;
}
```

Output:

```
$ ./lab_3
Received from parent: Hello, child!
```

> The process ended successfully. Wasn't it?

Yes! But only the parent process does. The child process is still hanging silently in the background:

```
$ ps -x | grep lab_3
  84505 pts/6    S      0:00 ./lab_3
```

Attach strace to its PID to debug:

```
$ sudo strace -f -tt -T -p 84505
strace: Process 84505 attached
09:53:09.756990 read(3
```

=> `read` syscall never returns a value => The process hangs forever

Trace back onto our code we can see that the pipe's write end in child process never get closed, causing `read()` to block forever.

Lab 4

Run Valgrind (memcheck) on a program with a deliberately introduced memory leak and a use-after-free bug; interpret the report and fix both

Mentioned in part 3

Revision #1
Created 2026-09-23 01:38:15 UTC by Dung Vu
Updated 2026-09-23 01:39:46 UTC by Dung Vu